

SESSION:-1

use("foodieApp")

switched to db foodieApp

db["restaurants"].find()

```
{
  _id: ObjectId('6a928edcdf870ec16ec755e0'),
  name: 'Punjab Grill',
  cuisine: 'North Indian',
  rating: 4.5,
  location: 'Connaught Place, New Delhi'
}
{
  _id: ObjectId('6a928ee6df870ec16ec755e1'),
  name: 'Toscano',
  cuisine: 'Italian',
  rating: 4.3,
  location: {
    city: 'Bengaluru',
    locality: 'Indiranagar'
  },
}
```

```
    menu: [
      'Margherita Pizza',
      'Penne Arrabbiata',
      'Tiramisu'
    ]
  }
  {
    _id: ObjectId('6a928ee6df870ec16ec755e2'),
    name: 'Brimstone BBQ',
    cuisine: 'Barbecue',
    rating: 4.7,
    location: {
      city: 'Mumbai',
      locality: 'Bandra'
    },
    menu: [
      'Smoked Ribs',
      'Pulled Pork Burger',
      'Cornbread'
    ]
  }
}
```

```

menu: [
  'Margherita Pizza',
  'Penne Arrabbiata',
  'Tiramisu'
]
}
{
  _id: ObjectId('6a928ee6df870ec16ec755e2'),
  name: 'Brimstone BBQ',
  cuisine: 'Barbecue',
  rating: 4.7,
  location: {
    city: 'Mumbai',
    locality: 'Bandra'
  },
  menu: [
    'Smoked Ribs',
    'Pulled Pork Burger',
    'Cornbread'
  ]
}

```

```

},
menu: [
  'Dim Sum',
  'Pad Thai',
  'Sushi Roll'
]
}
{
  _id: ObjectId('6a964b24b79e4505d2993ca3'),
  test: 'abc'
}
{
  _id: ObjectId('6a964b8eb79e4505d2993ca4'),
  test: 'test'
}
{
  _id: ObjectId('6a964e06b79e4505d2993ca5'),
  test: 'abc'
}
foodieApp> |

```

SESSION:-2

1. use insta_clone_db

Output: switched to db insta_clone_db)

2. db.createCollection("posts")

(Output: { ok: 1 })

```

db.stories.insertOne({
  username: "johndoe",
  image_url: "https://example.com",

```

```

  timestamp: new Date()

```

})

4. `db.stories.drop()`

(Output: true)

5. *show collections*

(Output: posts)

6. `db.dropDatabase()`

(Output: { dropDatabase: "insta_clone_db", ok: 1 })

7. **show dbs**

```
use insta_clone_db
switched to db insta_clone_db
db.createCollection("posts")
{ ok: 1 }
db.stories.insertOne({
  username: "johndoe",
  image_url: "https://example.com",
  timestamp: new Date()
})
{
  acknowledged: true,
  insertedId: ObjectId('6a966791218db090564bf165')
}
db.stories.drop()
true
show collections
posts
```

```
> db.dropDatabase()
< { ok: 1, dropped: 'insta_clone_db' }
> show dbs
< Mp2026      180.00 KiB
   admin      148.00 KiB
   config     108.00 KiB
   foodieApp   80.00 KiB
   local       40.00 KiB
insta_clone_db >
```

SESSION:-3

```
> db["CRUD Operations"].find()
<
> db.restaurants.insertOne({
  name: "Burger Hub",
  cuisine: "American",
  location: "Surat, Gujarat",
  rating: 4.5
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a966afd0441229d73cb66ec')
}
> db.movies.insertMany([
  {
    title: "Kalki 2898 AD",
    genre: "Sci-Fi",
    releaseYear: 2024,
    imdbRating: 7.4
  },
  {
```

```
    {
      title: "Stree 2",
      genre: "Horror-Comedy",
      releaseYear: 2024,
      imdbRating: 7.0
    },
    {
      title: "Deadpool & Wolverine",
      genre: "Action",
      releaseYear: 2024,
      imdbRating: 7.7
    }
  ])
  {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6a966b0e0441229d73cb66ed'),
      '1': ObjectId('6a966b0e0441229d73cb66ee'),
      '2': ObjectId('6a966b0e0441229d73cb66ef')
    }
  }
}
```

```

> db.flipkart_products.find({ price: { $lt: 1000 } })
<
> db.flipkart_products.find(
  { price: { $lt: 1000 } },
  { name: 1, price: 1, _id: 0 }
)
<
> db.restaurants.updateOne(
  { name: "Burger Hub" },
  { $set: { rating: 4.7 } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
> db.orders.deleteMany({ status: "cancelled" })
< {
  acknowledged: true,
  deletedCount: 0
}

```

1. Insert one restaurant document
2. Insert three movie documents
3. Find products under ₹1000
4. Update Burger Hub's rating
5. Delete cancelled orders

SESSION:-4

```
> use("Mp2026")
< switched to db Mp2026
> db["Spotify playlist"].find(
❌ • SyntaxError: Unexpected token (1:28)

> 1 | db["Spotify playlist"].find(
    |                               ^
> db.playlists.insertMany([
  {
    name: "Hot Hits India",
    likes: 3200,
    genre: "Pop",
    followers: 8500
  },
  {
    name: "Desi Hip-Hop Central",
    likes: 1800,
    genre: "Hip-Hop",
    followers: 6200
  },
  {
    name: "Chill Lo-Fi Beats",
    likes: 950,
    genre: "Lo-Fi",
    followers: 1200
  },
  {
    name: "Bollywood Butter",
    likes: 4500,
    genre: "Pop",
    followers: 12000
  },
  {
    name: "Workout Bangers",
    likes: 1500,
    genre: "Hip-Hop",
    followers: 3400
  },
  {
    name: "Acoustic Mornings",
    likes: 400,
    genre: "Acoustic",
    followers: 800
  }
])
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a966e701fe0b2bc8c97e080'),
    '1': ObjectId('6a966e701fe0b2bc8c97e081'),
    '2': ObjectId('6a966e701fe0b2bc8c97e082'),
    '3': ObjectId('6a966e701fe0b2bc8c97e083'),
    '4': ObjectId('6a966e701fe0b2bc8c97e084'),
    '5': ObjectId('6a966e701fe0b2bc8c97e085')
  }
}
db.playlists.find(
  { followers: { $gt: 1000 } },
  { name: 1, followers: 1, _id: 0 }
)
{
  name: 'Hot Hits India',
  followers: 8500
}
{
  name: 'Desi Hip-Hop Central',
  followers: 6200
}
{
  name: 'Chill Lo-Fi Beats',
  followers: 1200
}
{
  name: 'Bollywood Butter',
  followers: 12000
}
{
  name: 'Workout Bangers',
  followers: 3400
}
db.playlists.find(
  { genre: { $in: ["Pop", "Hip-Hop"] } }
).sort({ likes: -1 })
{
  _id: ObjectId('6a966e701fe0b2bc8c97e083'),
  name: 'Bollywood Butter',
  likes: 4500,
  genre: 'Pop',
  followers: 12000
}
```

```

> db.playlists.find(
  { likes: { $gte: 500, $lte: 2000 } }
).sort({ followers: -1 }).limit(3)
< {
  _id: ObjectId('6a966e701fe0b2bc8c97e081'),
  name: 'Desi Hip-Hop Central',
  likes: 1800,
  genre: 'Hip-Hop',
  followers: 6200
}
{
  _id: ObjectId('6a966e701fe0b2bc8c97e084'),
  name: 'Workout Bangers',
  likes: 1500,
  genre: 'Hip-Hop',
  followers: 3400
}
{
  _id: ObjectId('6a966e701fe0b2bc8c97e082'),
  name: 'Chill Lo-Fi Beats',
  likes: 950,
  genre: 'Lo-Fi',
  followers: 1200
}
> db.playlists.find(
  {
    followers: { $gt: 5000 },
    likes: { $gte: 1000 }
  },
  { name: 1, genre: 1, _id: 0 }
)
< {
  name: 'Hot Hits India',
  genre: 'Pop'
}
{
  name: 'Desi Hip-Hop Central',
  genre: 'Hip-Hop'
}
{
  name: 'Bollywood Butter',
  genre: 'Pop'
}
}
Mp2026 >

```

```

> db.playlists.find(
  { genre: { $in: ["Pop", "Hip-Hop"] } }
).sort({ likes: -1 })
< {
  _id: ObjectId('6a966e701fe0b2bc8c97e083'),
  name: 'Bollywood Butter',
  likes: 4500,
  genre: 'Pop',
  followers: 12000
}
{
  _id: ObjectId('6a966e701fe0b2bc8c97e080'),
  name: 'Hot Hits India',
  likes: 3200,
  genre: 'Pop',
  followers: 8500
}
{
  _id: ObjectId('6a966e701fe0b2bc8c97e081'),
  name: 'Desi Hip-Hop Central',
  likes: 1800,
  genre: 'Hip-Hop',
  followers: 6200
}
{
  _id: ObjectId('6a966e701fe0b2bc8c97e084'),
  name: 'Workout Bangers',
  likes: 1500,
  genre: 'Hip-Hop',
  followers: 3400
}
}

```

1. Insert 5+ playlist documents
2. Playlists with more than 1000 followers (projected)
3. Pop or Hip-Hop playlists, sorted by likes descending
4. Likes between 500–2000, top 3 by followers
5. Trending playlists (followers > 5000 AND likes ≥ 1000)

SESSION:-5

```
> use("Mp2026")
< switched to db Mp2026
> db["Evening Vibes"].find()
<
> db.playlists.updateOne(
  { name: "Evening Vibes" },
  { $set: { genre: "Chill" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
> db.restaurants.updateMany(
  { offerExpiryDate: { $lt: new Date() } },
  { $unset: { temporaryOffer: "" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
> db.playlists.updateOne(
  { _id: 101 },
  { $push: { songs: "Calm Waters" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
> db.products.updateOne(
  { sku: "F12345" },
  { $addToSet: { tags: "trending" } }
)
```

```
> db.products.updateOne(
  { sku: "F12345" },
  { $addToSet: { tags: "trending" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
> db.products.updateOne(
  { name: "Wireless Earbuds", stock: { $gt: 0 } },
  { $inc: { stock: -1 } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
> db.products.find()
<
Mp2026 > |
```

1. \$set — update genre for 'Evening Vibes'
2. \$unset — remove expired 'temporaryOffer' field
3. \$push — add a song to an existing array
4. \$addToSet — add 'trending' tag without duplicates
5. \$inc — decrease stock, never below zero

SESSION:-6

```
> use("Mp2026")
< switched to db Mp2026
> db["Aggregation Framework."].find()
<
> db.orders.aggregate([
  {
    $match: {
      orderDate: {
        $gte: new Date(new Date().setDate(new Date().getDate() - 30))
      }
    }
  }
])
<
> db.orders.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$userId",
      totalSpent: {
        $sum: { $multiply: ["$items.price", "$items.quantity"] }
      }
    }
  },
  { $sort: { totalSpent: -1 } }
])
<
> db.restaurants.aggregate([
  { $match: { cuisine: "Cafe", city: "Ahmedabad" } },
  { $sort: { rating: -1 } },
  { $limit: 3 }
])
<
> db.movies.aggregate([
  {
    $group: {
      _id: "$genre",
      totalCollection: { $sum: "$boxOffice" }
    }
  },
  {
    $project: {
      _id: 0,
      genre: "$_id",
      totalCollection: 1
    }
  }
])
```

```
])
< {
  totalCollection: 0,
  genre: 'Sci-Fi'
}
{
  totalCollection: 0,
  genre: 'Action Thriller'
}
{
  totalCollection: 0,
  genre: 'Epic Action'
}
{
  totalCollection: 0,
  genre: 'Action'
}
{
  totalCollection: 0,
  genre: 'Horror-Comedy'
}
> db.payments.aggregate([
  {
    $match: {
      paymentDate: {
        $gte: new Date(new Date().setDate(new Date().getDate() - 7))
      }
    }
  },
  {
    $group: {
      _id: {
        $dateToString: { format: "%Y-%m-%d", date: "$paymentDate" }
      },
      totalSales: { $sum: "$amount" },
      transactionCount: { $sum: 1 }
    }
  },
  { $sort: { _id: 1 } },
  {
    $project: {
      _id: 0,
      date: "$_id",
      totalSales: 1,
      transactionCount: 1
    }
  }
])
```

```

    }
  }
})
<
> db.payments.find()
<
> db["Aggregation Framework"].find()
<
> [
  { date: "2026-08-26", totalSales: 45200, totalTransactions: 38 },
  { date: "2026-08-27", totalSales: 52100, totalTransactions: 41 },
  { date: "2026-08-28", totalSales: 38900, totalTransactions: 29 },
  { date: "2026-08-29", totalSales: 61500, totalTransactions: 47 },
  { date: "2026-08-30", totalSales: 44300, totalTransactions: 35 },
  { date: "2026-08-31", totalSales: 58700, totalTransactions: 44 },
  { date: "2026-09-01", totalSales: 22100, totalTransactions: 18 }
]
< [
  { date: '2026-08-26', totalSales: 45200, totalTransactions: 38 },
  { date: '2026-08-27', totalSales: 52100, totalTransactions: 41 },
  { date: '2026-08-28', totalSales: 38900, totalTransactions: 29 },
  { date: '2026-08-29', totalSales: 61500, totalTransactions: 47 },
  { date: '2026-08-30', totalSales: 44300, totalTransactions: 35 },
  { date: '2026-08-31', totalSales: 58700, totalTransactions: 44 },
  { date: '2026-09-01', totalSales: 22100, totalTransactions: 18 }
]
Mp2026 >

```

1. \$match — orders placed in the last 30 days
2. \$group — total amount spent per user
3. Top 3 highest-rated cafes in Ahmedabad
4. Total box office collection per genre
5. Daily sales report — last 7 days

SESSION:-7

```
> use("Mp2026")
< switched to db Mp2026
> db["Indexing"].find()
<
> db.restaurants.insertMany([
  { name: "Pasta Paradise", cuisine: "Italian", location: "Ahmedabad", rating: 4.5 },
  { name: "Bella Italia", cuisine: "Italian", location: "Surat", rating: 4.2 },
  { name: "Dragon Wok", cuisine: "Chinese", location: "Ahmedabad", rating: 4.0 },
  { name: "Spice Route", cuisine: "Indian", location: "Vadodara", rating: 4.6 },
  { name: "Pizza Mania", cuisine: "Italian", location: "Surat", rating: 4.3 },
  { name: "Golden Dragon", cuisine: "Chinese", location: "Surat", rating: 3.9 },
  { name: "Curry House", cuisine: "Indian", location: "Rajkot", rating: 4.1 },
  { name: "Mama Mia", cuisine: "Italian", location: "Ahmedabad", rating: 4.7 },
  { name: "Wok This Way", cuisine: "Chinese", location: "Ahmedabad", rating: 4.4 },
  { name: "Roman Table", cuisine: "Italian", location: "Vadodara", rating: 4.0 },
  { name: "Punjabi Tadka", cuisine: "Indian", location: "Surat", rating: 4.5 }
])

db.restaurants.find({ cuisine: "Italian" })
< {
  _id: ObjectId('6a9684a31b8ebfa1643d16f3'),
  name: 'Pasta Paradise',
  cuisine: 'Italian',
  location: 'Ahmedabad',
  rating: 4.5
}
{
  _id: ObjectId('6a9684a31b8ebfa1643d16f4'),
  name: 'Bella Italia',
  cuisine: 'Italian',
  location: 'Surat',
  rating: 4.2
}
{
  _id: ObjectId('6a9684a31b8ebfa1643d16f7'),
  name: 'Pizza Mania',
  cuisine: 'Italian',
  location: 'Surat',
  rating: 4.3
}
```

```
{
  _id: ObjectId('6a9684a31b8ebfa1643d16fa'),
  name: 'Mama Mia',
  cuisine: 'Italian',
  location: 'Ahmedabad',
  rating: 4.7
}
{
  _id: ObjectId('6a9684a31b8ebfa1643d16fc'),
  name: 'Roman Table',
  cuisine: 'Italian',
  location: 'Vadodara',
  rating: 4
}
> db.restaurants.find({ cuisine: "Italian" }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Mp2026.restaurants',
    parsedQuery: {
      cuisine: {
        '$eq': 'Italian'
      }
    },
    indexFilterSet: false,
    queryHash: '6703AD7D',
    planCacheShapeHash: '6703AD7D',
    planCacheKey: 'ADA507DE',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        cuisine: {
          '$eq': 'Italian'
        }
      }
    },
    nss: 'Mp2026.restaurants',
    direction: 'forward'
  },
  rejectedPlans: []
}
```

```
    },
    executionStats: {
      executionSuccess: true,
      nReturned: 5,
      executionTimeMillis: 0,
      totalKeysExamined: 0,
      totalDocsExamined: 13,
      executionStages: {
        isCached: false,
        stage: 'COLLSCAN',
        filter: {
          cuisine: {
            '$eq': 'Italian'
          }
        },
      },
      nReturned: 5,
      executionTimeMillisEstimate: 0,
      works: 14,
      advanced: 5,
      needTime: 8,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'Mp2026.restaurants',
      direction: 'forward',
      docsExamined: 13
    }
  },
  queryShapeHash: 'C5CCC3AD8CDC0E8491F1BF1E9F33A92609664E9AD89065B1258394B542D42B62',
  command: {
    find: 'restaurants',
    filter: {
      cuisine: 'Italian'
    },
    '$db': 'Mp2026'
  },
  serverInfo: {
    host: 'LAPTOP-05AMEJP7',
    port: 27017,
    version: '8.3.0',
    gitVersion: '35e8c8a57f78157ed9fac1a9e90ee6c1818adab6'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
  }
}
```

```

serverInfo: {
  host: 'LAPTOP-05AMEJP7',
  port: 27017,
  version: '8.3.8',
  gitVersion: '35e8c8a57f78157ed9fac1a9e90ee6c1818adab6'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
> db.restaurants.createIndex({ cuisine: 1 })
< cuisine_1
> db.restaurants.find({ cuisine: "Italian" }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Mp2026.restaurants',
    parsedQuery: {
      cuisine: {
        '$eq': 'Italian'
      }
    },
    indexFilterSet: false,
    queryHash: '6703AD7D',
    planCacheShapeHash: '6703AD7D',
    planCacheKey: '1FE15A55',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      nss: 'Mp2026.restaurants',

```

```
inputStage: {
  stage: 'IXSCAN',
  nss: 'Mp2026.restaurants',
  keyPattern: {
    cuisine: 1
  },
  indexName: 'cuisine_1',
  isMultiKey: false,
  multiKeyPaths: {
    cuisine: []
  },
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'forward',
  indexBounds: {
    cuisine: [
      '["Italian", "Italian"]'
    ]
  }
},
rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 5,
  executionTimeMillis: 7,
  totalKeysExamined: 5,
  totalDocsExamined: 5,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 5,
    executionTimeMillisEstimate: 0,
    works: 6,
    advanced: 5,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    nss: 'Mp2026.restaurants',
    docsExamined: 5,
```

```
alreadyHasObj: 0,
inputStage: {
  stage: 'IXSCAN',
  nReturned: 5,
  executionTimeMillisEstimate: 0,
  works: 6,
  advanced: 5,
  needTime: 0,
  needYield: 0,
  saveState: 0,
  restoreState: 0,
  isEOF: 1,
  nss: 'Mp2026.restaurants',
  keyPattern: {
    cuisine: 1
  },
  indexName: 'cuisine_1',
  isMultiKey: false,
  multiKeyPaths: {
    cuisine: []
  },
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'forward',
  indexBounds: {
    cuisine: [
      '["Italian", "Italian"]'
    ]
  },
  keysExamined: 5,
  seeks: 1,
  dupsTested: 0,
  dupsDropped: 0,
  peakTrackedMemBytes: 0
}
},
```

```
queryShapeHash: 'C5CCC3AD8CDC0E8491F1BF1E9F33A92609664E9ADB9065B125B394B542D42B62',
command: {
  find: 'restaurants',
  filter: {
    cuisine: 'Italian'
  },
  '$db': 'Mp2026'
},
serverInfo: {
  host: 'LAPTOP-05AMEJP7',
  port: 27017,
  version: '8.3.8',
  gitVersion: '35e8c8a57f78157ed9fac1a9e90ee6c1818adab6'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
```

```
> db.restaurants.createIndex({ cuisine: 1, location: 1 })

db.restaurants.find({ cuisine: "Chinese", location: "Ahmedabad" }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Mp2026.restaurants',
    parsedQuery: {
      '$and': [
        {
          cuisine: {
            '$eq': 'Chinese'
          }
        },
        {
          location: {
            '$eq': 'Ahmedabad'
          }
        }
      ]
    },
    indexFilterSet: false,
    queryHash: 'B8D71C09',
    planCacheShapeHash: 'B8D71C09',
    planCacheKey: '5FAD6965',
    optimizationTimeMillis: 10,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      nss: 'Mp2026.restaurants',
      inputStage: {
        stage: 'IXSCAN',
        nss: 'Mp2026.restaurants',
        keyPattern: {
          cuisine: 1,
          location: 1
        },
      },
      indexName: 'cuisine_1_location_1',
      isMultiKey: false,
      multiKeyPaths: {
        cuisine: [],
```

```
      location: []
    },
    isUnique: false,
    isSparse: false,
    isPartial: false,
    indexVersion: 2,
    direction: 'forward',
    indexBounds: {
      cuisine: [
        '["Chinese", "Chinese"]'
      ],
      location: [
        '["Ahmedabad", "Ahmedabad"]'
      ]
    }
  }
},
rejectedPlans: [
  {
    isCached: false,
    stage: 'FETCH',
    filter: {
      location: {
        '$eq': 'Ahmedabad'
      }
    },
    nss: 'Mp2026.restaurants',
    inputStage: {
      stage: 'IXSCAN',
      nss: 'Mp2026.restaurants',
      keyPattern: {
        cuisine: 1
      },
      indexName: 'cuisine_1',
      isMultiKey: false,
      multiKeyPaths: {
        cuisine: []
      },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
```

```
    keyPattern: {
      cuisine: 1,
      location: 1
    },
    indexName: 'cuisine_1_location_1',
    isMultiKey: false,
    multiKeyPaths: {
      cuisine: [],
      location: []
    },
    isUnique: false,
    isSparse: false,
    isPartial: false,
    indexVersion: 2,
    direction: 'forward',
    indexBounds: {
      cuisine: [
        '["Italian", "Italian"]'
      ],
      location: [
        '[MinKey, MaxKey]'
      ]
    },
    keysExamined: 5,
    seeks: 1,
    dupsTested: 0,
    dupsDropped: 0,
    peakTrackedMemBytes: 0
  }
},
queryShapeHash: 'C5CCC3AD8CDC0E8491F1BF1E9F33A92609664E9ADB9065B1258394B542D42B62',
command: {
  find: 'restaurants',
  filter: {
    cuisine: 'Italian'
  },
  '$db': 'Mp2026'
},
serverInfo: {
  host: 'LAPTOP-05AMEJP7',
  port: 27017,
  version: '8.3.8',
  gitVersion: '35e8c8a57f78157ed9fac1a9e90ee6c1818adab6'
},
```

```
    indexBounds: {
      cuisine: [
        '["Italian", "Italian"]'
      ],
      location: [
        '["MinKey", "MaxKey"]'
      ]
    }
  },
  rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 5,
  executionTimeMillis: 0,
  totalKeysExamined: 5,
  totalDocsExamined: 5,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 5,
    executionTimeMillisEstimate: 0,
    works: 6,
    advanced: 5,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    nss: 'Mp2026.restaurants',
    docsExamined: 5,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 5,
      executionTimeMillisEstimate: 0,
      works: 6,
      advanced: 5,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'Mp2026.restaurants',
```

```
    keyPattern: {
      cuisine: 1,
      location: 1
    },
    indexName: 'cuisine_1_location_1',
    isMultiKey: false,
    multiKeyPaths: {
      cuisine: [],
      location: []
    },
    isUnique: false,
    isSparse: false,
    isPartial: false,
    indexVersion: 2,
    direction: 'forward',
    indexBounds: {
      cuisine: [
        '["Italian", "Italian"]'
      ],
      location: [
        '[MinKey, MaxKey]'
      ]
    },
    keysExamined: 5,
    seeks: 1,
    dupsTested: 0,
    dupsDropped: 0,
    peakTrackedMemBytes: 0
  }
},
queryShapeHash: 'C5CCC3AD8CDC0E8491F1BF1E9F33A92609664E9ADB9065812583948542D42B62',
command: {
  find: 'restaurants',
  filter: {
    cuisine: 'Italian'
  },
  '$db': 'Mp2026'
},
serverInfo: {
  host: 'LAPTOP-05AMEJP7',
  port: 27017,
  version: '8.3.8',
  gitVersion: '35e8c8a57f78157ed9fac1a9e90ee6c1818adab6'
},
```

```
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
```

1. Insert 10 restaurants and query Italian cuisine
2. Add index on 'cuisine' and compare execution time
3. Create the index:
4. After creating the index:
5. Compound index on 'cuisine' and 'location'
6. Drop the cuisine index and observe performance